

From CASHomon model sets to feature importance

Contents

A set of good models	1
Training, selection and explanation data	1
Search and verify the model set	1
Calculate PFI for every retained model	2
Compare feature reliance	2

A set of good models

`cashomon` searches a finite pool of algorithms and hyperparameter settings using `TruVaRImp`, Algorithm 1 in [Ewald et al. \(2026\)](#). Its goal is to find several configurations with similar predictive performance. Comparing their explanations shows how much feature reliance can vary among those models.

The numerical search works without the optional ML packages:

```
domain <- data.frame(x = seq(0, 1, length.out = 12))
search <- truvarimp(domain, function(row) (row$x - 0.3)^2,
  epsilon_abs = 0.05, noise = 0, budget = 8)
search
#> TruVaRImp: 12 candidates; 4 evaluations
#> L (inside): 5 H (outside): 7 U (uncertain): 0 M (possible best): 1
#> Stopped: classified
```

L, H and U contain indices predicted inside, outside and unresolved with respect to the implicit loss threshold. M contains potential minimizers. These are surrogate classifications; `fit_cashomon()` verifies the fitted models before returning the empirical set.

Training, selection and explanation data

Reserve three disjoint sets of rows: training for fitting candidates, validation for selecting near-optimal models, and a holdout for PFI. This prevents explanation data from influencing membership.

The following sections require `mlr3`, `rpart`, `xplainfi` and `R6`. They run when these optional packages are installed; otherwise their code remains visible.

```
task <- mlr3::as_task_regr(mtcars, target = "mpg")
set.seed(42)
rows <- sample(task$row_ids)
train <- rows[1:16]
validation <- rows[17:24]
test <- rows[25:32]
```

This small dataset demonstrates the API. Use larger, appropriately stratified or grouped partitions for substantive analysis.

Search and verify the model set

```
pool <- cashomon_candidates(
  learners = list(Tree = mlr3::lrn("regr.rpart"),
    Baseline = mlr3::lrn("regr.featureless")),
  grids = list(Tree = list(cp = c(0.001, 0.01, 0.05), maxdepth = c(2L, 4L)))
)
```

```
models <- fit_cashomon(task, pool, train, validation,
  epsilon_rel = 0.20, budget = 5, verify_all = TRUE)
models
models$members
```

The empirical threshold is the best evaluated validation loss multiplied by $1 + \text{epsilon_rel}$, plus `epsilon_abs`. All returned fitted models satisfy it. `verify_all = TRUE` evaluates the remaining pool after active search, making membership exact for this small finite pool. These extra fits are counted in `n_verification` and are outside the search budget.

For larger pools, `verify_all = FALSE` verifies unfitted members of `L` and uses the already evaluated candidates. Unobserved configurations may then be missing from the returned set, or may have a better loss than the empirical reference. Neither mode guarantees population-risk performance.

Calculate PFI for every retained model

```
pfi <- cashomon_pfi(models, test_rows = test, n_repeats = 30, seed = 42)
head(pfi$importance)
summarise_cashomon_pfi(pfi, by_class = TRUE)
```

`pfi$importance` has one row per fitted model and feature. `pfi$scores` retains each permutation. The adapter preserves the actual selected predictor during `xplainfi` computation, including stochastic fits.

Compare feature reliance

```
plot_cashomon_pfi(pfi, "cloud")
```

Each point is a model's mean PFI. The bands show the full range across retained models and diamonds mark the median. Use `permutation_intervals = TRUE` to add within-model permutation percentiles; these are not confidence intervals.

```
plot_cashomon_pfi(pfi, "heatmap")
```

The heatmap groups model profiles by class and orders them by validation loss. Negative importance values remain visible.

```
plot_cashomon_pfi(pfi, "ranks")
```

Ranks show whether models agree on the most important features. Tied importances receive average ranks.

```
plot_cashomon_pfi(pfi, "performance")
```

The performance view compares PFI against validation loss, with the membership threshold marked. Each returned plot is a ggplot object that can be customized and saved with `ggplot2::ggsave()`.

Across-model ranges describe only the retained finite set. Permutation variation and variation across models are distinct; permutations are averaged within each model before summarizing across models. Marginal PFI can also be affected by correlated features and unusual combinations produced by permutation.

The installed regression example adds linear models and random forests and exports tables and PNG/PDF plots:

```
source(system.file("examples", "workflow.R", package = "cashomon"))
```